Reproducibility Failures in Quantized Model Evaluation: Item Ordering Moves 4-Bit Accuracy and Leaves `bf16` Exactly Unchanged

Anonymous ACL submission

Abstract

When a paper reports that 4-bit quantization costs two points of accuracy, that number is one score minus another, so any part of the evaluation that penalises the two models unequally ends up inside the reported gap. We evaluate three instruction-tuned models at five precisions, holding engine, decoding and prompts fixed, and vary properties of the evaluation apparatus that no protocol we are aware of reports. Two results are established by intervention or by a pre-registered contrast that survives multiple-comparison correction. First, item submission order changes 4-bit accuracy by up to 1.09 points of standard deviation across 10 orderings and leaves `bf16` *exactly* unchanged, with the engine's batch-invariant mode enabled: that mode's guarantee does not cover the quantized matrix-multiply kernels, and disabling them removes the variation for GPTQ and AWQ. The GPTQ result replicates on a second GPU architecture. Second, the extraction rule changes the measured quantization effect with a task-dependent sign, moving GPTQ W4A16 relative to `bf16` by +1.73 points on BBH and by -5.21 on GSM8K, so no single correction restores comparability. Three supporting results bound the picture: generation is reproducible within a process and diverges on up to 45% of items across processes, for a cause we do not identify; a fixed generation budget truncates quantized outputs more often without a resolvable accuracy cost; and a log-likelihood control on ARC reverses the sign of the delta on one of three models. We give remedies with measured costs and release 105,950 generations with per-item termination metadata and a runnable order-invariance probe.

1 Introduction

When a paper reports that 4-bit quantization costs two points of accuracy, that number was not measured. It is the difference between two measurements: the model scored at full precision, and the model scored after quantization. Anything about the evaluation that penalises those two runs by the same amount cancels when you subtract. Anything that penalises one more than the other does not, and lands in the reported gap instead. This paper asks what those unequal factors are, and establishes two that no evaluation protocol we are aware of reports.

The framing matters because part of the answer is already known. Kurtic et al. (2025) showed that reported degradation had been overstated and attributed the excess to untuned quantization-side hyperparameters. We treat that as established and prior. Our concern is the residual that survives quantization-side tuning, and our mechanisms are orthogonal to theirs: they act on the evaluation, not on the quantized model.

Changing every score is not the same as changing the gap between scores. That distinction organises the paper. It is well known that lenient answer extraction raises absolute scores (Jo et al., 2025; Yu et al., 2025), and that prompt formatting moves them substantially (Sclar et al., 2024). What has not been established is whether such choices change the *gap* between a full-precision model and a quantized one, which is the number papers actually report. A choice that lowers every model's score by the same amount is a nuisance and nothing more, because the gap between them survives it. A choice that lowers some models' scores more than others changes that gap, and so contaminates every published comparison that did not report the choice. We test for the second kind, and find it.

Two results, each established by intervention or by pre-registered contrast. First, serving the same items through the same engine and varying only their submission order changes 4-bit accuracy by up to 1.09 points of standard deviation across 10 orderings, while leaving `bf16` accuracy *exactly*

unchanged under both extraction rules. This happens with the engine's batch-invariant mode enabled, because that mode's guarantee does not cover the quantized matrix-multiply kernels the engine selects by default. Disabling them drives the variation to zero on 2 of the quantized settings, so the mechanism is established by intervention rather than inferred. Second, how strictly the answer is extracted interacts with precision, and the interaction changes sign with the task: it moves GPTQ W4A16 relative to `bf16` by +1.73 points on BBH and by -5.21 on $\text{GSM8K}_{\text{CoT}}$. Preregistered contrasts confirm that it differs across tasks and across methods. If it were uniform, a single correction would remove it. Because it is not, no protocol-level adjustment recovers a comparable number, and the only remedy is to report the choice.

Three supporting results, which bound the picture rather than explain it. Generation is bitwise reproducible *within* a job and not *across* jobs, so the standard remedy of repeating a measurement in one process cannot detect the divergence; we refute four candidate causes but do not identify the real one. Under a fixed generation budget, quantized settings lose their answers to the cutoff more often, but the resulting accuracy change does not survive correction, so we report it as a bound. And a log-likelihood control on ARC, where extraction should have nothing to distort, leaves the delta unchanged on two of three models and reverses its sign on the third, which is direct support for the second result.

This is not an argument for scoring more permissively. Two facts prevent it. Ren et al. (2026) find that 8.2–44.1% of answers graded correct are reached by shortcut hacking, so leniency overcredits. And we measure that lenient extraction also *destroys* correct answers, at a small but nonzero rate, so it is not a monotone relaxation of strict extraction but a different selection rule with two-sided error. The contribution is to *calibrate* extraction, not to loosen it.

The literature's centre of gravity runs against us, and that belongs here. The dominant finding in this area is that benchmarks *understate* quantization damage: Marchisio et al. (2024) report a 1.7% automatic drop in Japanese corresponding to 16.0% by human judgement, and Dutta et al. (2024) show that models matched on accuracy still diverge behaviourally. We regard both directions as simultaneously true, because they are different quantities. An accuracy delta can over-penalise formatting *and* under-detect behavioural change. Nothing here licenses a conclusion that quantization is benign; the claim is about the comparability of reported numbers, not about deployment risk.

Contributions. (i) Item ordering moves 4-bit accuracy and leaves `bf16` exactly unchanged, with GPTQ W4A16 replicated on a second GPU architecture, with the mechanism pinned by intervention to quantized kernels that the engine's batch-invariant mode does not cover, and four gaps identified in that mode. (ii) The precision-by-scoring interaction is real, changes sign across tasks and differs across methods, so no single correction recovers comparability. (iii) Three supporting results: a cross-process reproducibility failure, with the four candidate mechanisms we refuted; a bounded null for the generation budget; and an ARC control showing that extraction can decide the direction of a delta even where the answer space is a single letter. (iv) Remedies with measured costs, rather than a diagnosis alone. Generating every compared precision inside a single process removes the cross-process variation at no cost; disabling the quantized matrix-multiply kernels removes the ordering variation for GPTQ and AWQ at roughly $3.7\times$ evaluation wall clock, and does nothing for NF4, whose mechanism is separate. Because a kernel flag is specific to one engine version, the recommendation we actually make is the *test* rather than the flag: verify order-invariance for whichever kernels your backend selected, by running the same items in several orderings and checking bitwise agreement. We release that probe. (v) Three fields a protocol should report, each because we measured it moving the quantized precisions and not the `bf16` baseline: the item submission order, whether the compared precisions were generated in the same process, and the generation budget together with its truncation rate. (vi) A corpus of 105,950 generations with per-item termination and length metadata, released so that any extractor can be re-evaluated without a GPU. (vii) An audit of our own extraction pipeline, which had silently degraded 2% of that corpus until a census found it, offered as an instance of the effect we measure rather than as an aside.

Reading this paper. §2 places the work against the two literatures it sits between, one on quantization degradation and one on evaluation-protocol sensitivity. §3 fixes the apparatus and includes an audit of our own extraction pipeline. §4 reports the two headline results first (§4.1 and §4.2), followed by four supporting results. §5 states what a reader should change, and the Limitations section states what we could not measure.

2 Related Work

A live contradiction about generation length. Whether low-bit quantization lengthens chain-of-thought output is currently disputed. Lian et al. (2026) report that INT4/INT3 quantization can preserve accuracy while *increasing* reasoning-token usage, and propose a CoT Token Inflation Ratio to quantify it; Lotfi et al. (2026) likewise find that aggressive post-training quantization lengthens CoT. Liu et al. (2025) report the opposite: "contrary to expectations, quantized models do not exhibit increased output lengths." We take no position on which generator behaviour is correct, because the disagreement has a consequence that neither side measures. Under a *fixed* generation cap, a model that emits longer traces is scored wrong more often whenever its answer falls beyond the cap. That is a scoring outcome, not a competence one. No study on either side of this dispute sweeps the generation cap, so no study can separate the two. That separation is our H1.

Degradation has already been partly explained, by other causes. Kurtic et al. (2025) show that reported quantization degradation was overstated, and attribute the excess to untuned quantization-side hyperparameters. We treat that result as established and *prior*, not as a target: our claim concerns the residual that remains after quantization-side tuning, and our mechanism is orthogonal to theirs. Lotfi et al. (2026) supply the most direct antecedent for the present work, finding that in as many as 52% of quantized-model failures the model reaches the correct answer in an intermediate step but does not emit it as the final answer. Their interest is in fixing the generator, via a logit penalty on overthinking markers; ours is in what such failures do to a reported accuracy delta when they are left in place.

The centre of gravity runs the other way. The dominant finding in this literature is that benchmarks *understate* quantization damage, not overstate it. Marchisio et al. (2024) find a 1.7% automatic drop in Japanese corresponds to 16.0% by human judgement, and Dutta et al. (2024) show that models matched on accuracy still diverge behaviourally (the "flips" framing, which is theirs and which we do not claim). We regard both directions as simultaneously true: an accuracy delta can over-penalise formatting *and* under-detect behavioural change, because these are different quantities. Our contribution is to the first, and it does not license any conclusion about the second.

Calibrating extraction, not loosening it. An argument that evaluation should simply be more permissive is already refuted. Ren et al. (2026) find that 8.2–44.1% of answers graded correct are reached by shortcut hacking, so lenient extraction over-credits as surely as strict extraction under-credits. We therefore treat the strict/permissive contrast as a measurement instrument to be characterised rather than a knob to be turned in one direction, and we report both extraction rules throughout.

Prompt- and protocol-sensitivity. Hua et al. (2025) ask whether prompt sensitivity is a genuine model property or an artifact of evaluation, and we are structurally the sequel: they vary the prompt, we vary the scorer, and both ask whether a reported effect survives the instrument. Their null result on MATH deserves emphasis rather than a footnote: extraction there already uses symbolic equivalence, so a well-engineered verifier leaves little for a protocol effect to explain. This bounds our claim rather than supporting it, and it is why our task selection targets benchmarks whose canonical extractors are regular expressions.

The nearest prior work, and what it deliberately designed out. Pape et al. (2026) is the closest study to our §4.1 and must be distinguished precisely. They survey 35,000 publications, find that only 36% of papers without released code name even their inference backend, and show that switching backend alone moves GSM8K by 16.6 points. We adopt their framing and their survey rather than repeating it. But their controlled comparison runs at *batch size one*, chosen explicitly to prevent batching-induced instability, holds precision at FP16, and varies only random seeds within an engine; batch invariance and quantized kernel selection do not enter. The instability they design

out is the one we make the object of study: we hold the engine, the weights and the decoding configuration fixed, vary only which items share a batch, and attribute the resulting divergence to a named quantized kernel by intervention. Their root-cause analysis independently supports our wider point: their two largest outliers were a spurious BOS injection and an unnoticed repetition penalty, worth 8.3 and 11.7 points, both configuration defaults rather than model properties.

Batch invariance, and where its guarantee stops. He and Thinking Machines Lab (2025) traced nondeterministic inference to reductions whose order depends on how many requests share a batch, and introduced batch-invariant kernels that remove the dependence; vLLM's batch-invariant mode, which we enable throughout, builds on that work. We do not revisit its result. We show that the guarantee does not extend to the quantized matrix-multiply kernels vLLM selects by default for 4-bit GPTQ and AWQ, so a quantized evaluation run in the deterministic mode is still order-sensitive while the same evaluation at `bf16` is not.

Sclar et al. (2024) establish that spurious formatting choices move scores substantially, and Jo et al. (2025) and Yu et al. (2025) show that answer extraction from reasoning traces is itself an unreliable step. Schaeffer et al. (2023) provide the methodological template we follow: a headline effect reattributed, in part, to the measurement instrument. The lm-evaluation-harness (Biderman et al., 2024) is our instrument, and its canonical filters are the object of study rather than an implementation detail.

Serving configuration is a confound we pin, not an axis we explore. Pape et al. (2026) find that inference-backend choice alone moves benchmark scores by up to 16.6 points, and Masoudian et al. (2026) attribute roughly 39% of out-of-the-box variance to it. We read this as a reason to freeze the engine, version, and every decoding parameter across all precision settings, which we do and report; a floating serving configuration would make precision effects unidentifiable. We claim no novelty on this axis.

Adjacent evaluations. Broad compression benchmarks (von Rad et al., 2026; Wang et al., 2025) and task-difficulty studies (Lee et al., 2025) establish the accuracy results we work against; Lee et al. (2025) in particular report that quantization disrupts instruction-following, which is the strongest available objection to our framing and which our compliance/competence split is designed to address directly rather than deflect. Prasad and Pal (2026) run a factorial quantization analysis, so factorial design is not our contribution; their outcome domain is safety alignment under sampling temperature, which is disjoint from ours. Kurt (2026) report the observation this paper sets out to explain (strict-versus-flexible GSM8K gaps of 41 to 64 points across fourteen `llama.cpp` quantizations) without interpreting the gap; we reproduce their numbers as an external anchor. Oladri et al. (2026) and Rasmussen and Qin (2026) report related failure-mode and scoring-rule effects, the former over FP32/FP16/NF4 only and so without bearing on GPTQ or AWQ.

What is new here. Permissive extraction is well known to raise absolute scores. What has not been established is whether it changes the quantization *difference*: an artifact in a level need not be an artifact in a delta, and the two are routinely conflated. We decompose the measured strict delta into competence, compliance, and truncation components, pre-register the decomposition before any generation exists, and report the residual precision effect that survives it.

3 Method

3.1 Experimental design

We evaluate three instruction-tuned models spanning a 3–8B size gradient: Qwen2.5-3B-Instruct, Phi-3.5-mini-instruct, and Llama-3.1-8B-Instruct. Each is pinned by commit SHA rather than tag, since tags move and reproducibility holds only within identical versions.

We compare five precision settings, which we call *arms*: `bf16`, GPTQ-8bit, GPTQ-4bit, AWQ-4bit, and bitsandbytes NF4. Five rather than two, because ordering them by bit width gives a graded series instead of a single before-and-after comparison. If accuracy falls steadily from `bf16` through 8-bit to 4-bit, that is much harder to explain as a measurement artifact, since an artifact of the evaluation has no reason to track bit width in order.

All manipulations are one-factor-at-a-time from a single fixed reference condition, so every reported effect is attributable to the one factor varied.

NF4 is included in the reference condition but excluded from the token-budget sweep, and we state the reason rather than the exclusion alone: NF4 has no CUDA kernels in our serving engine, and it is quantized *in flight* at load rather than loaded from a pre-made checkpoint. That is a backend difference layered on the bit-width difference, which we name explicitly rather than absorb.

3.2 Quantization and calibration

This is the most likely attack surface on the paper, so we state it in full. Both GPTQ and AWQ are produced by one tool, through one calibration code path, at *one* normalised budget: 512 sequences of at most 2048 tokens drawn from a single pinned shuffle of `ultrachat_200k`, identical for every arm, every bit-width, and every model. We report the realised token count rather than the product of those two numbers, since sequences shorter than the cap contribute their own length.

Normalisation is necessary because the reference implementation's own examples differ by **8**× in calibration budget between the two methods (GPTQ at 512×2048 symmetric; AWQ at 256×512 asymmetric). Accepting those defaults would mean a GPTQ-versus-AWQ difference confounded with an 8× calibration difference *and* a symmetry difference. We therefore normalise AWQ to symmetric 4-bit weight-only quantization, which is a deliberate deviation from its published example, and we flag it as such.

Group size is 128 and activation reordering is disabled for every GPTQ arm, matching the vendor's own released checkpoints. Disabling reordering is set explicitly rather than left to the library, whose default silently applies a static reordering. Every parameter above is hashed into a configuration digest carried on each generation record, so a reader can verify that no arm drifted.

All arms are self-quantized with identical settings. Vendor checkpoints are used only as an external cross-check, never mixed into the main matrix, because a widely used third-party GPTQ checkpoint for one of our models enables activation reordering against the vendor's own setting, which is exactly the kind of silent confound this design exists to exclude.

3.3 The scoring ladder

Every generation is stored once and scored many times. We apply a ladder of extractors to the *same* stored text, so the entire ladder costs zero additional generation and every step sees byte-identical inputs. We call each step a *rung*, and refer to them by name rather than by number throughout:

1. **strict**: the evaluation harness's canonical regular expression, used verbatim.
2. **strict-last**: the same expression selecting the *last* match instead of the first, which separates answer *position* from extraction *strictness*.
3. **permissive**: the harness's flexible extractor, with its rules enumerated in the pre-registration.
4. **symbolic**: equivalence checking rather than string matching.
5. **judge**: a trained answer verifier (Chen et al., 2025).

The headline contrast is strict versus permissive. Both are deterministic, auditable, and judge-independent, which is what makes the judge a robustness check rather than a load-bearing dependency. That distinction is doing real work here rather than being a precaution: we validated the strict and permissive rungs against human labels (Appendix E), but the judge itself was never scored against them, so we report it as an unvalidated extractor and rest no claim on it. The strict-last variant exists because a harness that takes the first match will score a model that restates its answer differently from one that does not, independent of correctness.

An audit of our own extractor, and what it found. Because this paper's claim is that unreported measurement choices move reported numbers, we audited our own extraction rather than assuming it. Our permissive extractor applied a commitment-phrase rule ahead of the harness's flexible-extract regex, and returned the committed answer with its unit attached (`"5 days"` against a gold of `"5"`). Because the harness's regex admits no alphabetic character, it could not have produced those values: the defect was ours, ordered ahead of published practice, and it returned worse answers than the harness would have on 2,068 of 105,950 generations. A full census of every rung-monotonicity violation is what found it; our unit tests passed throughout, because the fixture set contained no generation that continued

past its committed answer. Two interaction estimates changed conclusion when it was repaired and one changed sign. All numbers reported here are post-repair. We record the episode because it is an instance of the effect this paper measures rather than an aside: a well-intentioned extractor improvement silently degraded 2% of a six-figure corpus, our unit tests passed throughout, and only an exhaustive audit surfaced it. It is also why we report a census of every affected item rather than a sampled estimate.

3.4 Compliance, competence, and truncation

We decompose the measured strict delta into three additive components: a *competence* component (the model does not know the answer), a *compliance* component (it knows but does not emit the answer in the expected form), and a *truncation* component (it would have emitted the answer but ran out of budget). Recovering the third requires two fields that cannot be reconstructed after the fact, the engine's termination reason and the generated token count, so both are recorded on every generation, and our writer refuses any record missing either.

Compliance is treated as a dependent variable in its own right, not merely as a correction term. We also distinguish an answer being *extractable* from the model having *committed* to it: a fallback rule that returns the last number in a truncated trace has extracted something the model never asserted, and counting that as compliance would inflate the very component we are trying to measure.

3.5 Serving configuration and determinism

The serving configuration is frozen and identical across every arm: one engine, one pinned version, one set of decoding parameters, greedy decoding throughout. This is not incidental rigour. Backend choice alone moves benchmark scores by up to 16.6 points (Pape et al., 2026) and accounts for roughly 39% of out-of-the-box variance (Masoudian et al., 2026), so a serving configuration that floated across arms would make precision effects unidentifiable regardless of sample size.

Greedy decoding is not by itself sufficient for reproducibility, because inference engines batch requests and batched reductions are not order-invariant. We therefore enable batch-invariant kernels (He and Thinking Machines Lab, 2025), run with tensor parallelism of one (batch invariance is broken above one), disable prefix caching (it changes greedy output and is explicitly outside the invariance guarantee, and it would otherwise help throughput considerably on our shared few-shot prefixes), and fix the maximum number of co-resident sequences to a value *measured* rather than assumed: the largest setting that is bitwise reproducible across repeated identical runs for *every* precision arm, which is a minimum over arms because a single shared value is required.

We report that value, the per-arm bitwise disagreement rates behind it, the GPU model, and the CUDA and driver versions. Where a quantized arm is less reproducible than `bf16`, the difference is a noise floor on every contrast involving that arm, and we pre-commit to claiming no effect smaller than it.

4 Results

The first two subsections are the headline results. They are established by intervention (§4.1) or by a pre-registered contrast that survives Holm–Bonferroni (§4.2). The four that follow are supporting results: an observation whose cause we bound but do not identify (§4.3), a bounded null (§4.4), and two results on the channels behind the second headline (§4.5, §4.6).

4.1 Item ordering moves 4-bit accuracy and leaves `bf16` unchanged

We serve every arm through a single engine version (0.26.0) with batch-invariant kernels enabled, tensor parallelism disabled, prefix caching off, and a fixed decoding configuration, then vary one parameter that no evaluation protocol we are aware of reports: *the order in which items are submitted*. Because requests are batched dynamically, an item's position determines which other items share its batch, and therefore the shapes over which the engine sums its intermediate results. Floating-point addition is not associative, so summing the same numbers in a different order can differ in the last bit, and in chain-of-thought generation a single differing token changes every token after it.

Holding the ordering fixed and running the same items again changes nothing: not one of the 15 model-and-precision combinations we measured produced a single differing output. So every difference reported below is caused by the ordering, not by run-to-run variation.

Aggregate accuracy moves, and only for quantized arms. Table 1 gives the standard deviation

	strict		permissive	
arm	SD	range	SD	range
BF16	0.00	0.00	0.00	0.00
GPTQ W4A16	0.87	2.80	0.75	2.40
AWQ W4A16	0.52	2.00	1.09	4.20
bnb NF4	0.11	0.40	0.10	0.20

Table 1: Accuracy across 10 orderings of the same 500 items, in accuracy points, measured at the served batch size $B = 32$ (§3.5). The standard deviation is the inferential quantity; the range is descriptive only, being unstable and monotone in the number of orderings sampled. `bf16` is identical under every ordering at both rungs.

	text divergence		accuracy SD	
arm	Marlin on	off	Marlin on	off
GPTQ W4A16	53.2	0.0	0.42	0.00
AWQ W4A16	53.6	0.0	0.35	0.00
bnb NF4	49.4	49.4	0.31	0.31

Table 2: Paired intervention at $B = 8$: identical inputs, identical engine, with and without the Marlin and Machete kernels. Text divergence and accuracy SD in percentage points. bitsandbytes NF4 is unaffected at exactly $1.00\times$ because it never used these kernels, so its ordering sensitivity has a second and distinct cause.

of accuracy across 10 orderings of the same 500 items at the served batch size we adopt. For `bf16` the accuracy is *identical* under all 10 orderings, at both scoring rungs: 2 of 2 cells show exactly zero variation. Every quantized arm moves, by up to 1.09 points of standard deviation.

Why we report a standard deviation and not a spread. A minimum–maximum spread is monotone in the number of orderings sampled, so it grows with effort rather than converging. We can show this rather than assert it: the same quantity measured over three orderings gave a spread of 1.6 points and over 10 gives 4.2, with no change to the underlying effect. Any spread reported over a handful of runs is therefore a lower bound on the spread its own protocol would have produced with more. The standard deviation, by contrast, stayed within a narrow band across arms and rungs.

Three properties matter more than the magnitude. The shift is **deterministic** given an ordering rather than sampling noise, so it cannot be averaged away by repeated runs. It is **bidirectional**: accuracy moves both up and down across orderings, so no ordering is the correct one and the effect cannot be calibrated away. And it is **attributable by intervention** rather than inferred, as follows.

The batch-invariant mode's guarantee does not cover Marlin or Machete. Disabling those two kernels through `VLLM_DISABLED_KERNELS`, changing nothing else, drives order-induced divergence to zero on 2 arms (Table 2), on generated text *and* on accuracy. These are the quantized matrix-multiply kernels vLLM selects by default, and they are not covered by its batch-invariant mode: the AWQ kernel path carries a batch-invariance guard, the GPTQ path carries none, and the sampler and kernel autotuning paths are likewise unguarded. We report this as a defect in the feature rather than in either kernel; AWQ has the guard and is nonetheless affected, so the guard as implemented is insufficient.

What this establishes, and what it does not. The intervention locates the mechanism in a specific pair of kernels and shows the effect is not merely correlated with them, because removing them removes it. It does not explain the arithmetic: we do not identify which reduction differs, in what order, or why AWQ's batch-invariance guard fails to hold. We claim the localisation, which is what an intervention buys, and not the explanation.

The effect replicates on a second GPU architecture. We repeated both designs on an NVIDIA A10G (Ampere, SM 8.6) with the same image, engine version, weights and probe. Across 10 orderings of 500 items at $B = 32$, `bf16` text differs on 0.0% of items and its accuracy does not move at either rung. GPTQ W4A16 text differs on 69.8% of items (71.2% on the L40S) and its strict accuracy has a standard deviation of 0.70 points (0.87 on the L40S). Disabling Marlin and Machete at $B = 8$, over the default three orderings, again takes text divergence from 53.4% to 0.0%, with the engine's log showing a fallback to a Triton kernel. The result is therefore not specific to one GPU or one architecture, although it remains specific to one engine version.

Two controls rule out the obvious alternatives. Enabling kernel autotuning leaves every figure bit-identical, so the effect is not an autotuning artifact. And bnb NF4 is unmoved by the intervention while remaining order-sensitive, so at least one further mechanism exists that is Marlin-independent; we note that it becomes *less* order-sensitive at

larger batch sizes, the reverse of every other arm, and leave the cause unresolved.[1]

The size of the effect depends on where you measure it. The level at which divergence is measured changes its apparent size by about $5\times$: generated text diverges on up to 59.4% of items, while scored correctness diverges on at most 11.2% at the same configuration, with the extracted answer between them. Reporting text divergence alone would substantially overstate the impact on any reported number, and reporting correctness alone would conceal how much of the trace is unstable, so we report all three levels at every batch size we swept (Appendix B).

Notably the permissive rung is the more order-sensitive of the two at the answer level, because it accepts the last number anywhere in the trace and the trace is what diverges, whereas strict extraction requires a committed marker that is comparatively stable. Leniency therefore buys recovery of truncated answers at the cost of exposure to batch-composition noise, which is an argument for calibrating extraction rather than loosening it (§3.3).

Scope.

Scope of these numbers. All figures in this section come from a single confirmatory run of 10 orderings at the batch size we adopt, with the terminator set the `chat` template requires (§3). An earlier exploratory run used three orderings and a terminator set that truncated most BigBench-Hard items, and we report the later run rather than pooling the two, because they measure different configurations. The pre-registration fixed the ordering count, the seeds and the batch size before this run executed (Appendix F).

4.2 The scoring rule changes the measured quantization effect, unevenly

The rungs differ in how much they credit a generation, so the quantity a reader compares across precisions depends on which rung produced it. For each arm we take the strict-to-permissive gap and subtract `bf16`'s gap on the same items: a difference of differences in accuracy points, bootstrapped over item clusters. Positive means leniency helps the quantized arm *more* than `bf16`, negative means less.

The interaction is real, and its sign depends on the task. Table 3 gives all 12 registered estimates. 5 exclude the null under Holm–Bonferroni within the family of 8. On BBH, leniency helps the 4-bit arms more than it helps `bf16`: GPTQ W4A16 +1.73 pp $[+0.79, +2.70]$ and AWQ W4A16 +1.55 pp $[+0.61, +2.52]$. On $\text{GSM8K}_{\text{CoT}}$ the same comparison reverses: GPTQ W4A16 -5.21 pp $[-6.77, -3.61]$ and `bnb` NF4 -8.44 pp $[-9.91, -6.95]$.

A single pooled estimate over values running from +1.73 to -8.44 would describe neither end, so we report the individual estimates and treat their disagreement as the finding.

The disagreement is tested, not observed. A sign that differs across cells could be sampling variation, so we pre-registered two contrasts on the difference between two interaction estimates, named structurally rather than chosen by magnitude, before computing either (Appendix F). Holding the arm fixed and varying the task gives +6.93 pp $[+5.09, +8.76]$; holding the task fixed and comparing GPTQ against AWQ at matched bit width gives -4.30 pp $[-5.84, -2.75]$. Both exclude zero and both reject under Holm–Bonferroni at adjusted levels of 0.025 and 0.050, corrected as their own family rather than pooled with the per-arm estimates, because *are two interactions different* is a different hypothesis over different quantities from *is this interaction non-zero*. The full contrast table is in Appendix C.

Why heterogeneity is the load-bearing part. A uniform interaction would be a nuisance with a remedy: report one rung, or apply one offset, and cross-paper comparison is restored. A heterogeneous one has no such remedy. The correction that would make GPTQ W4A16 comparable on BBH has the wrong sign on $\text{GSM8K}_{\text{CoT}}$, and the correction for GPTQ does not serve AWQ at the same bit width. So the practical recommendation is not a better default but a disclosure requirement: the extraction rule belongs in the reported protocol, because no post-hoc adjustment recovers it.

Where the interaction is absent, both cases are informative. GPTQ W8A16 is null on both generative tasks (+0.06 pp, +0.53 pp), which is what a dose-response predicts: the arm that degrades

[1] `bitsandbytes` is proposed for out-of-tree maintenance in vLLM RFC #39583, on performance grounds. Two explanations are consistent with our data: a data-dependent reduction whose result concentrates at larger batch sizes, or a different kernel-selection path entirely. We do not have evidence to separate them.

	ARC-C (gen.) (n=1,500)		BBH (n=3,300)		GSM8K$_{\text{CoT}}$ (n=3,957)	
arm	est.	95% CI	est.	95% CI	est.	95% CI
GPTQ W8A16	+0.00†	[-0.51, +0.51]	+0.06	[-0.67, +0.76]	+0.53	[-0.33, +1.42]
GPTQ W4A16	+0.00†	[-0.51, +0.51]	**+1.73**	[+0.79, +2.70]	**-5.21**	[-6.77, -3.61]
AWQ W4A16	+0.00†	[-0.51, +0.51]	**+1.55**	[+0.61, +2.52]	-0.91	[-2.50, +0.66]
bnb NF4	+0.00†	[-0.51, +0.51]	**-0.97**	[-1.91, -0.06]	**-8.44**	[-9.91, -6.95]

Table 3: Precision × scoring interaction in accuracy points against `bf16`, under strict versus permissive extraction, on 105,950 generations. **Bold** excludes the null. † marks a worst-case bound rather than a bootstrap interval: ARC-C's estimate is *exactly* zero, so there is nothing to resample and the interval is the Wilson zero-event bound (§4.6).

least gives the scoring rule least to disagree about. ARC-C under generative scoring is *exactly* zero for all four arms, bounded within ± 0.51 pp at n=1,500, which is the structural control of §4.6 behaving as predicted and is read as a bound rather than as evidence of no effect. We pre-committed to reporting this interaction whichever way it came out, since a null would have meant that extraction choices move levels without moving deltas and that published comparisons are safe despite unreported protocols. They are not.

One excluded arm, reported as a demonstration only. GSM8K under the hash-marker prompt (gsm8k) is excluded from the family above and from the Holm correction, because instruction-tuned models rarely emit the #### marker unprompted, so strict extraction there measures marker compliance rather than competence and a floored metric has no variance for an interaction to appear in (Appendix F). Its estimates are large and negative (GPTQ W4A16 -13.12 pp, AWQ W4A16 -7.91 pp), and we show them as the extreme end of the marker-naturalness gradient rather than as a test.

4.3 Reproducibility holds within a process and fails across them

The natural defence against measurement noise is to run it twice, and here that defence fails in a specific and instructive way. Within a single process, generation is bitwise reproducible: we observe zero differing items across 42,380 paired comparisons. Across process boundaries, the same configuration on the same hardware diverges on up to 45% of items for one model, and moves its accuracy by over a point. So a practitioner who repeats a measurement in one process obtains perfect agreement and concludes the pipeline is deterministic, which is exactly the wrong conclusion, reached by exactly the check a careful person would run. Reproducibility has to be assessed across job boundaries, and we know of no protocol that does so.

The mechanism we did not find, and what we ruled out. We report the cross-process effect without a mechanism, and we state what was eliminated so that the gap is bounded rather than open. Four candidates were tested and refuted. Differing KV-cache block counts from per-invocation memory profiling: refuted, because the engine-reported cache sizes are byte-identical between jobs, cell for cell. Batching scheduled by memory rather than by sequence count: refuted, because one arm is stable at a concurrency of $12.36\times$ while another diverges at $10.70\times$, so the two are not separable. Weight provenance, base repository against local checkpoint: refuted, because it predicts one model exactly and is contradicted by the other two. Sampling configuration, including a hidden repetition penalty read from each model's own generation config: refuted, because the model shipping the most aggressive defaults is the only one that is perfectly reproducible. What remains untested is state that is compiled or cached once per process, which is consistent with the within-process zero, and we identify it as the next hypothesis rather than claiming it.

4.4 A fixed generation budget converts trace length into degradation, but the accuracy cost is bounded

We submitted the same 12,600 items at three generation budgets, the harness default and two and four times it, and joined them on item identity across the reference run and the sweep. The budget is treated as an independent variable rather than an implementation detail, and the three levels were fixed as absolute token counts before any result existed.

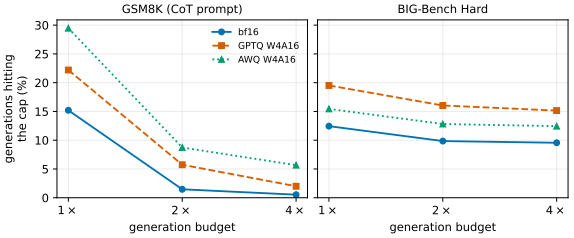


Figure 1: Generations stopped by the budget rather than by the model, at three budgets. Quantized arms are truncated more often at the default budget on both tasks, and raising the budget removes most of that excess on GSM8K. This is H1's mechanism; its accuracy consequence is reported below and is not statistically distinguishable from zero.

The mechanism is real and differential by precision. Figure 1 shows the fraction of generations that stop because they reach the cap rather than because the model finishes. At the default budget the quantized arms are truncated more often than `bf16` on both tasks, and raising the budget removes most of that excess on GSM8K: `bf16` falls from 15.2% to 0.5%, GPTQ from 22.2% to 2.0%, and AWQ from 29.5% to 5.7%. The quantized arms begin with more to gain and gain more of it, which is the behaviour H1 predicts.

The token counts say the same thing from the other side. On BIG-Bench Hard the GPTQ arm discards 415 tokens per generation after its scored answer at the largest budget, against 215 for `bf16`: the quantized model keeps writing past the answer we score, and a fixed cap therefore cuts it off more often.

The accuracy consequence is bounded, and we do not claim it. Measured degradation against `bf16` moves in the predicted direction on three of the four quantized task-arm pairs and against it on one, and *none* of the four changes survives Holm–Bonferroni over the four-member family: the smallest p is 0.048 against an adjusted threshold of 0.0125. The largest movement is AWQ on GSM8K at -1.80 pp (95% CI -3.60 to -0.02).

We report this as a bound rather than as an effect. At these item counts the data exclude a shrinkage larger than roughly three accuracy points and are consistent with none at all. The honest reading is that the truncation channel exists and is large when measured in tokens, and that at the budgets practitioners actually use it does not move the reported quantization delta by an amount we can resolve. That is a narrower claim than H1 was framed to make, and it is the one the data support.

Quantization does lengthen chain-of-thought. This settles a disagreement in the literature that neither side measured under a swept cap (Lian et al., 2026; Liu et al., 2025). On GSM8K at the default budget the mean scored answer is 169 tokens for `bf16`, 173 for GPTQ W4A16 and 180 for AWQ W4A16, and the ordering is the same on the full generation. The effect is modest and it is in the direction the inflation reports claim, not the direction of the null report.

4.5 Quantization shifts whether the model answers at all

Quantization changes not only whether an answer is right but whether the model commits to one at all. For each precision we cross its per-item outcome against `bf16`'s on the same item, giving a 2×2 table: committed under both, committed only under `bf16`, committed only under the quantized setting, or neither. The two off-diagonal counts are what matter, because they move in opposite directions and a net figure reports only their difference.

The net figure hides most of what happens. 48 of 108 per-cell comparisons exclude the null, so commitment genuinely shifts. But the direction is not consistent, and the clearest case is GPTQ W4A16 on BBH: 423 items lose their committed answer and 421 gain one, a net of -0.06 pp across 844 changed items. A paper reporting only the net would describe that setting as leaving commitment untouched. The largest net movements run both ways, from +2.73 pp for AWQ W4A16 on ARC-C to -9.88 pp for bnb NF4 on $\text{GSM8K}_{\text{CoT}}$.

This is the channel that explains the negative interactions. §4.2 found that permissive extraction sometimes helps `bf16` more than it helps a quantized setting, which is initially counterintuitive: leniency ought to rescue the weaker model. The transitions supply the reason. Where a quantized setting gains committed answers relative to `bf16`, it has already collected at strict extraction much of what leniency would otherwise recover, so there is less left for the permissive rule to add. Compliance and extraction are not independent channels, and treating them as separable is what makes the interaction look paradoxical.

What we do not claim here. We report the transitions and stop. The registered decomposition

	ARC-C			BBH			$GSM8K_{CoT}$			$GSM8K_{hash}$*		
arm	chg.	net	$\neq 0$	chg.	net	$\neq 0$	chg.	net	$\neq 0$	chg.	net	$\neq 0$
GPTQ W8A16	4	**+0.27**	1	391	+0.03	0	342	**-1.01**	1	330	**-1.11**	2
GPTQ W4A16	84	**+2.53**	2	844	**-0.06**	5	990	**+2.58**	3	1031	**+6.04**	2
AWQ W4A16	99	**+2.73**	2	861	**+2.21**	9	1009	**-0.88**	1	1308	**+8.39**	3
bnb NF4	62	**+1.47**	1	670	**-2.06**	11	1511	**-9.88**	2	1051	**+9.73**	3

Table 4: Compliance transitions against `bf16` on the same items, pooled over the three models, at strict extraction. *chg.* is the number of items that changed commitment state in either direction; *net* is the resulting change in commitment rate in percentage points; $\neq 0$ counts how many of the underlying per-model cells exclude the null under an exact McNemar test. **Bold** marks a net with at least one such cell. *$GSM8K_{hash}$ is shown as a demonstration and excluded from the registered family (§4.2).

of the measured difference into competence, compliance and truncation components requires the always-compliant stratum, and that stratum falls below half the paired items in 14 of 27 cells, with three items in the worst case. We therefore report the decomposition as supplementary, alongside Manski bounds that are wide enough in those cells to be uninformative, and we do not rest any claim in this paper on it. The transitions above need no stratum, which is why they carry this section instead.

4.6 Even a constrained answer space does not protect the delta

ARC-Challenge asks for one of four letters, so extraction should have nothing to distort. We tested that directly. The same 500 items and the same prompt were scored two ways: by generating an answer and parsing a letter from it, and by comparing the log-likelihood of the four letters after the prompt's answer prefix, which removes the extraction step entirely. Weights, engine and hardware are identical between the two.

The registered quantity is the quantization delta under generation minus the same delta under log-likelihood, paired by item. On Phi-3.5 and Llama-3.1 it is small, at most 1.8 points in magnitude, and no estimate survives Holm–Bonferroni. On Qwen2.5 it is large: -13.2 points for GPTQ W4A16 (95% CI -16.6 to -10.0) and -8.4 for AWQ W4A16, the only 2 of 6 contrasts that survive correction.

The mechanism is visible in the generations. Qwen's `bf16` model often answers in prose rather than with a letter, writing for example "The best way to separate salt from water is with the use of heat", and the harness's terminator stops it at the first full stop, before any letter appears. Its quantized variants do this less. So under generation `bf16` scores 67.6% and its GPTQ variant 74.4%, which makes quantization look beneficial; under log-likelihood they score 80.2% and 73.8%, and the sign of the delta reverses.

This is the opposite of what the control was registered to show, and we report it as such. It is also the strongest instance of the paper's thesis: on the most constrained task we tested, the direction of a reported quantization effect is decided by whether the evaluation has an extraction step. The within-generation interaction of §4.2 was exactly zero on ARC because strict and permissive extraction fail on the same items, which is why a single scoring mode could not see this.

5 Discussion

What to do: two controllable factors, and one that must be reported. The three factors divide, and the division is principled rather than a matter of how much effort we spent on each. Item ordering and the process boundary are *nuisance parameters*: they have no bearing on the question anyone is asking, so the right response is to pin them, and we measured what that costs. Generating every compared precision inside one process removes the cross-process variation entirely and costs nothing, which follows from the exact zero over 42,380 paired comparisons in §4.3. Disabling the quantized matrix-multiply kernels removes the ordering variation for GPTQ and AWQ at roughly $3.7\times$ evaluation wall clock, which is a cheap trade for a one-off benchmark run and would not be a sensible one for production serving. It does nothing for NF4, whose ordering sensitivity survives the disable and whose mechanism we did not identify.

The extraction rule is different in kind. It is not a nuisance parameter but a *choice* about what counts as a correct answer, and §4.2 shows its ef-

fect varies by quantization method and by task. A quantity that varies with the thing you are measuring cannot be standardised away, so it has to be reported. That is why our recommendation is asymmetric: pin the first two, report the third.

What none of this licenses is a rule of the form "report accuracy to one decimal place" or "average over k runs". Averaging cannot help with a factor that is deterministic given a configuration, and numerical precision was never the problem.

Recommend the test, not the flag. A kernel name and an environment variable are specific to one engine version, and ours will age. The durable form of the same advice is a test: run the same items in several orderings and check whether the outputs agree bitwise, for whichever kernels your backend selected. That question stays meaningful when the kernel names change, and it is what we release alongside the corpus. Our own use of it is the worked instance rather than the recommendation itself.

Why the scoring rule must be reported rather than corrected. The heterogeneity in §4.2 is why every recommendation here is a disclosure requirement rather than an adjustment: the sign of the scoring effect depends on which method and which task a paper used, so there is no single correction to apply.

On auditing one's own instrument. The largest single error we found was our own. An extraction rule we had added ahead of the harness's returned worse answers than the harness would have, on 2,068 of 105,950 generations, and our unit tests passed throughout because no fixture exercised a generation that continued past its committed answer. Two of our interaction estimates changed conclusion when it was repaired and one changed sign. We report this because it is the same phenomenon as the rest of the paper observed from the inside, and because it suggests that a census over every item is worth more than a fixture set when the quantity of interest is a small differential.

6 Conclusion

Two unreported properties of an evaluation move reported quantization degradation, and both are established by intervention or by pre-registered contrast. Item submission order changes 4-bit accuracy while leaving `bf16` exactly unchanged, through quantized kernels that the engine's batch-invariant mode does not cover. The extraction rule interacts with precision with a sign that depends on the task and the method, so no single correction recovers comparability and the rule has to be reported rather than adjusted for. The process boundary and the generation budget belong in a protocol as well, although we could not identify the first's cause or resolve the second's accuracy cost. We release the full generation corpus so that any extraction rule can be re-evaluated without a GPU, including ours, which auditing found to be defective.

Limitations

The reproducibility claims are single-engine. The ordering result holds on two GPUs of different architectures (L40S, SM 8.9, and A10G, SM 8.6; §4.1), but for one engine version and one CUDA and driver pair, all of which we report. Batch invariance is version-specific and kernel-specific, so the ordering result should not be read as a claim about other engine versions. The cross-job result was measured on the L40S only and should not be read as a claim about other hardware. We also cannot serve a stock AWQ checkpoint in `bfloat16` on this engine version at all, which is itself a comparability limitation rather than only an inconvenience: published AWQ numbers are not dtype-comparable to GPTQ unless dtype is reported.

The cross-job mechanism is undiagnosed. We establish that generation is bitwise reproducible within a job and not across jobs, and we report four refuted hypotheses with the measurements that refuted them. We do not identify the cause. The finding is therefore an observation about what a third party will encounter, not an explanation, and our recommendation is correspondingly procedural rather than a fix.

The competence decomposition rests largely on bounds. The always-compliant stratum falls below half the paired items in 14 of 27 cells, and in the worst case (Qwen on the hash-marker GSM8K arm) it is three items. We therefore report the point estimate and the Manski bounds together for every cell, with the stratum size as a first-class number, rather than presenting a point estimate whose subgroup is too small to support it. Wide bounds are reported as wide. The decomposition is a supporting analysis in this paper, not a headline.

We do not report a counted survey of protocol reporting. Our pre-registration includes an audit computing, for each published degradation figure we cite, the confidence interval implied by that paper's own evaluation size. We verified four sources and stopped. Reporting four of fourteen would invite the question of why those four, and the honest answer would be that they were the four easiest to verify, which is not a selection rule a reader should accept. We therefore make the general claim on Pape et al. (2026)'s survey of 35,000 publications, which finds that only 36% of papers without released code name even their inference backend, and we cite our four verified cases as instances rather than as a rate.

Two registered hypotheses are not reported, and we name them rather than omit them. Our pre-registration includes two hypotheses we do not report. H2, the rate at which a correct answer appears in a generation's trace without reaching final answer position, is implemented and its false-positive guard is registered, but it is only measurable on records produced after a mid-study change to where the terminator is applied, and we have not recomputed it. H5, the same precision-by-scoring interaction under zero-shot prompting, has no instrument: our generation stages are fixed to the few-shot variant. In both cases a partial number would have been available and we judged that reporting one would be worse than reporting none, because a half-measured hypothesis invites the reader to weight it as though it were measured. They remain in the pre-registration, which is the record that they were planned rather than added or dropped to suit a result.

Scope deliberately excluded, and what that costs. There is no factual-recall task: TriviaQA was cut, so every task here is a reasoning task whose canonical extractor is a regular expression, and the structural control is the ARC mode pairing rather than a second domain. Our main matrix is self-quantized throughout, at a normalised calibration budget, which isolates the recipe but means we do not speak to arbitrary published checkpoints. Three models cannot support a variance estimate, so model is a fixed effect and no claim generalises to a model population. Finally, our anchor comparison against official checkpoints tests comparable quality at matched settings rather than reproduction, because the official checkpoints were calibrated on data we do not have; one of its three arms failed its pre-registered band, and we report the failure and the grounds on which we proceeded.

Ethics Statement

This work evaluates existing open-weight models on existing public benchmarks and introduces no new model, no new dataset of human origin, and no deployment. We see three ethical considerations worth stating.

Licensing, and one model we can run but not redistribute. Every model and dataset is used within its licence. Two cases need naming rather than assuming. Qwen2.5-3B-Instruct is released under the Qwen research licence rather than Apache-2.0, and we report it as such. Our primary judge is released under CC-BY-NC-ND-4.0, which permits us to run it and report numbers but forbids redistributing the weights or any derivative, so the artifact release contains our judging code and its outputs and not the model. We name a second, Apache-2.0 judge for exactly this reason, so that the reliability claim can be reproduced by someone unwilling to accept the first licence.

Human validation of the extraction rungs. We validated the automated extraction rungs against 250 human labels drawn from `pilot-v2`, stratified over 48 cells of model, precision and task, with 120 items labelled twice for agreement. Inter-annotator agreement was 0.975 raw (95% CI 0.929 to 0.991), Cohen's κ 0.943 and Gwet's AC1 0.968; we report AC1 as primary because the label distribution is skewed and κ's chance term penalises exactly that imbalance. The three disagreements were resolved by applying the written rubric as a rule lookup, with the governing clause recorded per item, and one of them resolved to a label neither annotator had chosen. Blinding was by omission rather than concealment: the precision condition, the parser's verdict and the judge's verdict are absent from the annotation file, item order was shuffled independently per annotator with published seeds, and no label was ever pre-filled. The rubric, the seeds, the labels and the adjudication record are released.

The LLM judge rung remains *unvalidated* and we state that rather than imply otherwise: we report no agreement statistic for it and no claim in this paper rests on it. The headline contrast is strict against permissive, both deterministic and

auditable.

What a paper about measurement error could be misused to argue. Our findings show that reported quantization degradation is in part an artifact of evaluation protocol. That could be read as licence to dismiss reported degradation generally, and we do not support that reading. The dominant finding in this literature runs the other way, with automatic metrics *understating* damage relative to human judgement, and our own results are about the *difference* between arms under a fixed protocol rather than about whether quantization is safe to deploy. We also report that leniency in extraction has two-sided error, so nothing here argues for scoring more permissively as a general remedy. The actionable content is that unreported measurement choices should be reported, which raises the cost of a claim rather than lowering it.

Compute is reported in full: every run, its duration and its cost are recorded in the released log, totalling roughly 35 GPU-hours on a single accelerator.

Acknowledgments

References

Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, and 11 others. 2024. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*.

Ding Chen, Qingchen Yu, Pengyuan Wang, Mengting Hu, Wentao Zhang, Zhengren Wang, Bo Tang, Feiyu Xiong, Xinchi Li, Chao Wang, Minchuan Yang, and Zhiyu Li. 2025. xverify: Efficient answer verifier for reasoning model evaluations. *arXiv preprint arXiv:2504.10481*.

Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is not all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Horace He and Thinking Machines Lab. 2025. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism.

Andong Hua, Kenan Tang, Chenhe Gu, Jindong Gu, Eric Wong, and Yao Qin. 2025. Flaw or artifact? rethinking prompt sensitivity in evaluating llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Hwiyeol Jo, Joosung Lee, Jaehone Lee, Sang-Woo Lee, Joonsuk Park, and Kang Min Yoo. 2025. Finding answers in thought matters: Revisiting evaluation on large language models with reasoning. *arXiv preprint arXiv:2510.14773*.

Uygar Kurt. 2026. Which quantization should i use? a unified evaluation of llama.cpp quantization on llama-3.1-8b-instruct. *arXiv preprint arXiv:2601.14277*.

Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. 2025. "give me bf16 or give me death"? accuracy-performance trade-offs in llm quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Jemin Lee, Sihyeong Park, Jinse Kwon, Jihun Oh, and Yongin Kwon. 2025. Exploring the trade-offs: Quantization methods, task difficulty, and model size in large language models from edge to giant. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*.

Xinyu Lian, Walid Krichene, Beichen Huang, Masahiro Tanaka, Olatunji Ruwase, Li Zhang, and Minjia Zhang. 2026. Quantization inflates reasoning: Token inflation as a hidden cost of low-bit reasoning models. *arXiv preprint arXiv:2606.25519*.

Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. 2025. Quantization hurts reasoning? an empirical study on quantized reasoning models. In *Conference on Language Modeling (COLM)*.

Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. 2026. Quantized reasoning models think they need to think longer, but they do not. *arXiv preprint arXiv:2606.00206*.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. 2024. How does quantization affect multilingual llms? In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Shahed Masoudian, Passant Shafaei, Monorama Swain, and Markus Schedl. 2026. What we observe as llm behavior can be a side-effect of inference backend. *arXiv preprint arXiv:2608.04714*.

Renuka Oladri, Mohan Vamsi Varadaraju Priya, and Jerry Wu. 2026. Silent failures in quantized llm reasoning: A taxonomy-based analysis of hollow convergence and failure mode shifts. *arXiv preprint arXiv:2607.09999*.

David Pape, Jonathan Evertz, and Lea Schönherr. 2026. The silent hyperparameter: Quantifying the impact of inference backends on llm reproducibility. *arXiv preprint arXiv:2605.19537*.

Hari Prasad and Ritam Pal. 2026. The joint effect of quantization and sampling temperature on llm safety alignment: A factorial analysis. *arXiv preprint arXiv:2606.29581*.

Anton Rasmussen and Hong Qin. 2026. When calibration depends on the scoring rule: Quantized biomedical llm classification. *arXiv preprint arXiv:2608.03854*.

Xuan Ren, Weiqi Zhai, Tianle Pu, Yihua Zhu, Hu Wei, and Bing Zhao. 2026. Right answer, wrong method: Shortcut hacking misleads the evaluation of llm reasoning on frontier science benchmarks. *arXiv preprint arXiv:2608.02442*. Work in progress.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.

Jonathan von Rad, Yong Cao, and Andreas Geiger. 2026. Unicomp: A unified evaluation of large language model compression via pruning, quantization and distillation. *arXiv preprint arXiv:2602.09130*.

Jiacheng Wang, Yejun Zeng, Jinyang Guo, Yuqing Ma, Aishan Liu, and Xianglong Liu. 2025. Slmquant: Benchmarking small language model quantization for practical deployment. *arXiv preprint arXiv:2511.13023*.

Qingchen Yu, Zifan Zheng, Shichao Song, Zhiyu Li, Feiyu Xiong, Bo Tang, and Ding Chen. 2025. xfinder: Large language models as automated evaluators for reliable evaluation. In *International Conference on Learning Representations (ICLR)*.

A Terms used in this paper

Every term here is also defined where it first appears in the text. This section is for looking one up out of order, not the definition of record.

arm One precision setting of one model, for example Llama-3.1-8B at GPTQ-4bit. We compare five.

`bf16` Brain floating point, 16 bits. The unquantized baseline every quantized setting is compared against.

GPTQ, AWQ Two post-training quantization methods. Both produce a weight file ahead of time, which the serving engine then loads.

bitsandbytes NF4 A third method, 4-bit NormalFloat. Unlike GPTQ and AWQ it is applied *in flight* when the model loads, rather than read from a prepared file, and it has no dedicated GPU kernels in our serving engine. Both facts matter: they are why the kernel remedy of §4.1 does not apply to it.

kernel The specialised GPU routine that performs one operation, here the quantized matrix multiplication. Different kernels compute the same mathematical result by different arithmetic routes, which is why the choice is observable.

Marlin, Machete The two quantized matrix-multiply kernels our engine selects by default for 4-bit GPTQ and AWQ. Disabling them is the intervention in §4.1.

batch invariance The property that a result does not depend on which other requests were processed alongside it. Our engine offers a mode intended to guarantee this; §4.1 reports where that mode does not reach.

strict, permissive The two answer-extraction rules whose disagreement this paper measures. *Strict* is the evaluation harness's canonical regular expression, used verbatim. *Permissive* is the harness's own flexible extractor, which accepts looser forms. Both are the harness's, not ours.

rung One step of the extraction ladder in §3.3. Five rungs run over the same stored text, so comparing them costs no additional generation.

interaction Here, the difference between how much permissive extraction helps a quantized setting and how much it helps `bf16` on the same items. Zero means the extraction rule is a nuisance; non-zero means it changes the reported gap.

the gap, the reported difference The number a paper reports as quantization degradation: the unquantized score minus the quantized score.

within a process, across processes Whether two compared measurements were produced by the same running instance of the serving engine, or by two separate runs. §4.3 shows this distinction is load-bearing.

B The order-divergence frontier in full

Table 5 reports order-induced divergence at every served batch size we measured.

C Heterogeneity contrasts in full

Both contrasts in §4.2 were pre-specified structurally, before either was computed, so magnitude cannot have selected them. The first holds the arm fixed and varies the task; the second holds the task fixed and compares GPTQ against AWQ at matched bit width. They are corrected as their own Holm–Bonferroni family, separate from the per-arm interaction estimates.

D Frozen configuration

Reproducibility of the results in §4.1 holds only within an identical software and hardware configuration, so the configuration is reported in full rather than summarised. Everything below is also machine-readable in the released artifact.

Models, pinned by commit rather than by tag. Tags move; commits do not.

Qwen2.5-3B-Instruct `aa8e72537993ba99e69dfaafa59ed015b17504d1`

Phi-3.5-mini-instruct `2fe192450127e6a83f7441aef6e3ca586c338b77`

Llama-3.1-8B-Instruct `0e9e39f249a16976918f6564b8830bc894c89659`

Serving. vLLM 0.26.0 with batch-invariant kernels enabled; `dtype` `bfloat16`; `max_model_len` 8192; `max_num_seqs` 32, set by the audit in §4.1 rather than chosen; tensor parallelism 1, since batch invariance does not hold above it; prefix caching disabled; asynchronous scheduling disabled; FlashInfer sampling disabled; kernel autotuning disabled.

Decoding. Greedy throughout: `temperature` 0, `top_p` 1, `top_k` disabled, all repetition and presence penalties 0, `seed` 0. Each model's own `generation_config.json` is deliberately *not* read, because two of our three models ship non-neutral sampling defaults there, which would make the decoding configuration differ by model in a study whose design holds the apparatus fixed.

Quantization. GPTQ and AWQ are produced with `llmcompressor`, calibrated on `HuggingFaceH4/ultrachat_200k` (`train_sft`), 512 sequences at 2048 tokens, shuffle seed 20260823, giving a ceiling of 1,048,576 calibration tokens. Group size 128 and activation reordering disabled on every GPTQ arm, matching the vendor's own released checkpoints. bitsandbytes NF4 takes no calibration, being applied in flight at load.

Hardware, and why it is a scope limit rather than a detail. One NVIDIA L40S, compute capability 8.9, 46,068 MiB usable; CUDA 13.0; driver 595.91.07; `torch` 2.11.0+cu130. The second-GPU replication of §4.1 ran on one NVIDIA A10G (SM 8.6) with the same container image tag, engine version (0.26.0, from the job log), CUDA 13.0 and driver 595.91.07. Batch invariance is implemented per architecture, so the exactly-zero `bf16` results are a claim about these two architectures and this engine version, not about GPUs in general.

E Annotation rubric

Two annotators labelled 250 generations from the pilot run, stratified across 48 cells of model, precision and task, as *correct*, *incorrect* or *no answer*; 120 items were labelled by both. The rubric fixes a decision procedure in order: find the answer the model presents as final, take the last commitment where there are several, compare by value rather than formatting, and apply one written rule to each of nine edge cases, among them truncation before versus after commitment, a correct number that appears only in the reasoning, a hedged pair of answers, and a reference answer that looks wrong. It was fixed, with a ten-item practice round completed, before any real label was recorded. The precision condition and both automated verdicts were absent from the annotation file rather than hidden in it, item order was shuffled independently per annotator with published seeds, and no label was pre-filled.

Agreement on the overlap was 0.975 raw (95% CI 0.929 to 0.991), Cohen's κ 0.943 and Gwet's AC1 0.968. The three disagreements were resolved by looking up the rubric clause that decides each item, never by deferring to either annotator, and one resolved to a label neither had chosen. Against these labels, strict extraction agrees with the human verdict on 76.9% of `bf16` items and 56.2% of 4-bit items, with false-negative rates of 23.1% and 41.1%, while permissive extraction agrees on 85.6% and 89.0% with false positives of

	$B = 8$			$B = 32$			$B = 64$		
arm	text	str.	perm.	text	str.	perm.	text	str.	perm.
BF16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
GPTQ W8A16	50.6	5.0	6.8	53.4	5.8	7.0	51.4	5.6	7.8
GPTQ W4A16	53.2	5.4	5.8	59.0	7.4	8.2	59.4	7.4	11.2
AWQ W4A16	53.6	4.2	7.6	54.8	5.6	9.4	54.8	5.2	9.6
bnb NF4	49.4	7.6	6.6	5.2	0.0	0.2	1.8	0.0	0.0

Table 5: Order-induced divergence at three measurement levels (generated text, and correctness under strict and permissive extraction) by served batch size B, on 500 items with 3 orderings. This bounds what a third party can reproduce at each configuration, which is why we report the whole frontier rather than only the value we adopt.

contrast	levels	est.	95% CI	n	α_{Holm}	decision
across tasks	GPTQ W4A16: BBH vs $\text{GSM8K}_{\text{CoT}}$	+6.93	[+5.09, +8.76]	7,257	0.025	**reject**
across arms	$\text{GSM8K}_{\text{CoT}}$: GPTQ W4A16 vs AWQ W4A16	-4.30	[-5.84, -2.75]	3,957	0.050	**reject**

Table 6: Heterogeneity of the precision $\times$ scoring interaction. Estimates are differences between two interaction estimates, in accuracy points, with a percentile cluster bootstrap over items. Contrast 1 is unpaired because the two tasks have different items; contrast 2 is paired because they share items, and resampling them independently would discard the correlation and overstate the interval.

at most 0.7%. The rubric, seeds, labels and adjudication record are released.

F Pre-registration

The pre-registration is released in full with the artifact. Summarised here is what it fixed before the corresponding data existed, since that is the property that matters.

Fixed before any generation. The three generation budgets as absolute token counts per task; the item counts and subsample seeds; the permissive extractor's rules, enumerated rather than described; the interaction model and the family within which multiple comparisons are corrected; the identification fix for the competence contrast in all three of its parts; and the rule that no effect smaller than the pipeline's own measured noise floor will be claimed.

Fixed before the run that tested it. The ordering count, the seeds and the batch size for the confirmatory ordering run of §4.1; the two heterogeneity contrasts of §4.2, named structurally so that magnitude could not have selected them; and the decision rule for the template robustness arm, recorded with both branches declared publishable before either was observed.

Registered after the relevant data was seen, and labelled as such. Several sections were added or amended after exploratory results existed. Each states that inline, with the exploratory numbers quoted, because discovery followed by registered confirmation is ordinary practice while presenting it as pre-data when the revision history shows otherwise is not. The clearest instances are the extraction rung added after monotonicity violations were measured, the batch size moved after a pre-registered escalation threshold was crossed, and the invocation-noise estimator, which registers an estimator rather than a number precisely because the pilot's figure was measured at the wrong sample size.

One threshold dissolved rather than lowered. A minimum stratum size had been registered as a proportion. It failed in most cells, and it was also the wrong shape, since a proportion ranks a large task's small fraction below a small task's large one. It was therefore removed in favour of reporting the point estimate and the bounds together in every cell, which is more informative and is not an analysis decision that could have been tuned to the result.